DOCUMENTAÇÃO INTERNA

Grafo

Guia do desenvolvedor: todas as funções, como usar e como o código está organizado

Este guia descreve o que o Grafo faz hoje (outubro de 2026), como cada recurso é usado e onde ele vive no código, além de como rodar, testar, publicar e configurar o Supabase.

thegrafo.vercel.app · repositório iago069/Grafo-project · Copyright © 2026 Iago Ariel Gularte Barbosa

Sumário

1. Visão geral e arquitetura
2. Conta, login e onde ficam os dados
3. Projetos e galeria
4. Editor de diagramas (Topologia)
5. Tipos de diagrama
6. Equipamentos, ícones e modelos
7. Cabos, portas e grupos
8. Editar em lista e painel lateral
9. Bayface de Racks e Datacenter
10. Documentos (manuais e procedimentos)
11. Exportação: PDF, PNG, SVG, JSON, CSV
12. Relatórios e etiquetas com QR code
13. Equipe, compartilhar e histórico
14. NetBox e status online (agente)
15. Paleta de comandos, atalhos e ajuda
16. Temas e aparência
17. Sem internet e app instalável
18. Estrutura do código
19. Rodar, testar e publicar
20. Supabase: configuração completa
21. Segurança
22. Limites e problemas comuns
23. Próximos passos sugeridos

1. Visão geral e arquitetura

O Grafo é um aplicativo web para **documentar redes de provedores**: diagramas de topologia, disposição física dos racks (Bayface), documentos técnicos e exportações para clientes. Roda inteiro no navegador, sem etapa de build: HTML, CSS e JavaScript puros.

Camada	Como funciona
Interface	`index.html` + `assets/css/styles.css`. Página inicial (login) e editor na mesma página.
Editor	`assets/js/editor/app.js`, **gerado** a partir de `src/app/00…12-*.js` com `npm run montar`. Expõe `window.GrafoApp` para os módulos.
Desenho	Viz.js (Graphviz em WebAssembly, via jsDelivr) gera o SVG; o editor reposiciona, roteia os cabos e aplica ícones.
Módulos	`assets/js/recursos/*.js`: racks, documentos, comandos (Ctrl+K), equipe, compartilhar, histórico, etiquetas, planilha, integração, primeiros passos, instalar.
Validação	`assets/js/comum/esquema.js` com Zod: todo dado que entra (formulário, JSON, CSV, nuvem) passa por aqui, nos modos `estrito` e `reparar`.
Conta e nuvem	Supabase (Auth + Postgres + Storage). `assets/js/conta/auth.js` faz login e sincronização.
Hospedagem	Vercel: estático + funções em `api/`. Cada envio ao `main` publica sozinho, se os testes passarem.
Rede interna	`agente/grafo-agente.js`: programa local que fala com o NetBox e dá ping, porque o site na nuvem não alcança a rede.

> **Regra de ouro**
> Para mudar o editor, edite `src/app/` e rode `npm run montar`. Nunca edite `assets/js/editor/app.js` à mão: o teste de montagem acusa se ele estiver desatualizado.

2. Conta, login e onde ficam os dados

- **Entrar / criar conta** com e-mail e senha (Supabase Auth). Recuperação de senha por e-mail.
- **Usar sem login**: os projetos ficam só no navegador; Compartilhar, Histórico, Equipe e envio de arquivos pedem conta (aparecem com cadeado).
- Os projetos ficam na tabela `grafo_user_data` (uma linha por pessoa, protegida por RLS) e são salvos automaticamente poucos segundos depois de cada mudança. O selo do topo mostra "Salvo na nuvem", "Alterações pendentes" ou "Sem internet".
- **Conflito**: se a mesma conta salvar em outra aba ou computador, o Grafo avisa e deixa escolher qual versão manter.
- **Excluir conta** (menu da conta) apaga a conta e os projetos, depois de digitar o e-mail.
- **Backup**: lembrete no topo quando o último backup tem 7 dias ou mais.

Código: `assets/js/conta/auth.js`, `sessao.js`; SQL em `supabase/schema.sql`.

3. Projetos e galeria

- **Pílula do topo**: seletor do projeto, botão de **galeria** (ícone de grade), `+` novo projeto, lixeira, Equipe e Histórico.
- **Novo projeto** pergunta o nome e o tipo de diagrama.
- **Galeria**: todos os projetos em cartões com miniatura (blocos na cor do tipo ligados pelos cabos), contagem de equipamentos, cabos e racks, filtro por nome e selo *ABERTO*. Clique abre o projeto. Também pela paleta: "Ver todos os projetos".
- Limite: 100 projetos por conta.

Código: galeria em `assets/js/recursos/comandos.js`; troca de projeto em `src/app/06-projetos-historico-e-backup.js`.

4. Editor de diagramas

- **Quadro**: arraste o fundo para mover, roda do mouse para zoom, barra com desfazer/refazer, zoom e ajustar à tela. No tema escuro o quadro vira um "papel escuro" (só na tela e na exportação desse tema).
- **Selecionar**: clique; Shift/Ctrl+clique soma à seleção; Ctrl+A seleciona tudo.
- **Arrastar** move o equipamento (ou toda a seleção). **Guias de alinhamento** verde-água aparecem e o equipamento encaixa na linha dos vizinhos; segure `Alt` para arrastar livre.
- **Ficha ao passar o mouse**: LED de status, tipo, modelo, IP, rede, grupo e cabos.
- **Duplo clique** (ou Enter/F2 no equipamento em foco) abre o **painel lateral** de edição, que desliza da direita sem esconder o diagrama.
- **Excluir** não pergunta: exclui na hora e mostra um aviso com **Desfazer** (6 s). Ctrl+Z também desfaz.
- **Copiar/colar/duplicar**: Ctrl+C, Ctrl+V, Ctrl+D.
- **Organizar** (menu de layout): Automático, Centro (estrela), Esquerda→Direita, Cima→Baixo, Grupos lado a lado, Grade por grupo.
- **Anotações**: notas coloridas livres no desenho (ferramenta de anotação).
- **Rastrear caminho**: mostra o caminho de cabos entre dois equipamentos.
- **Buscar** por nome ou IP no campo do painel (Enter próximo, Shift+Enter anterior).
- **Legenda** embaixo: tipos com cor própria e quantidade de cada um.
- Limites: 35 equipamentos por diagrama, 2000 cabos.

Código: `src/app/01` (estado e vista), `02` (posições), `03` (cabos, grupos, arraste, guias), `04` (busca e caminho), `05` (zoom, modais, editor), `10` (geração do DOT e renderização).

5. Tipos de diagrama

Tipo	O que muda
Topologia	Nome, modelo, tipo e IP em cada equipamento; foco em portas e cabos.
Diagrama de redes	IP em destaque e o campo **VLAN / rede**.
Monitoramento	Status em destaque (ativo, manutenção, offline), legenda por status, cabos para equipamentos offline ficam vermelhos e tracejados.
Bayface de Racks	Troca o quadro pelo editor físico de racks (seção 9). O diagrama lógico continua guardado.

O seletor fica no título do painel (menu com ícone e descrição em Chrome/Edge; lista simples em Firefox/Safari). A troca tem transição animada.

6. Equipamentos, ícones e modelos

- **Adicionar equipamento** (menu lateral): nome, tipo, modelo, IP, grupo, status e ícone.
- **Ícones**: escolhidos pelo tipo/nome ou manualmente; catálogo em `assets/images/equipamentos/` e lista em `SIMBOLOS` de `src/app/00-catalogo.js`. Também emoji ou imagem própria. Opção de **cor do ícone**. Após trocar imagens, rode `npm run icones`.
- **Modelos**: `data/equipamentos.json` (lista "modelos" com fabricante, tipo e portas em faixas, ex.: `ether1-24`). Com modelo preenchido, os campos de porta viram lista com portas livres primeiro.
- **Cores por tipo** fixas para os comuns (OLT, ONU, Switch, Roteador, Rádio, DIO, Servidor, DWDM, Nobreak, Firewall, Storage...) em `corPorTipo` (`src/app/08`).

7. Cabos, portas e grupos

- **Adicionar cabeamento**: origem/destino, portas, tipo (Fibra, Cobre...), capacidade (1G, 10G, 100G, GPON...), cor, estilo e espessura.
- As portas aparecem nas laterais voltadas para o vizinho; com muitas portas o equipamento cresce. Aviso ao reutilizar uma porta já em uso.
- **Grupos** (campo Grupo / rack / sala) viram blocos com fundo no desenho.

8. Editar em lista e painel lateral

- **Editar em lista** (menu Diagrama): uma linha por equipamento com marca colorida do tipo, nome editável, tipo/modelo/IP embaixo e status em ponto; a seta abre os detalhes (ícone, modelo, IP, grupo, VLAN, cores, imagem). O mesmo para cabos.
- **Painel lateral**: o editor de equipamento/cabo desliza da direita; botão *Documentos deste equipamento* leva aos manuais ligados a ele.

9. Bayface de Racks e Datacenter

Escolha **Bayface de Racks** no tipo de diagrama. Três colunas: racks + biblioteca, palco do rack, propriedades.

Racks

- **+ Rack** cria um rack de 42U (alturas comuns de 12 a 48U). Propriedades: nome, local, **fileira**, patrimônio, fabricante, modelo, status, observações.
- **Fileiras**: preencha "Fileira" (A, B...). O próximo rack nasce na mesma fileira com nome sequencial (B1, B2...). A lista agrupa por local e fileira.

Equipamentos no rack

- **Biblioteca**: 17 categorias (servidor, switch, roteador, storage, firewall, OLT, DIO, patch panel, organizador, bandejas, PDU, nobreak, DWDM, telecom, painel cego, personalizado) e **busca de modelos reais** (82 modelos, 79 com foto real do painel, extraídas do drawio do Datacenter Ibirubá).
- Arraste da biblioteca para uma U, ou clique numa U livre e escolha. Arraste o equipamento para mover; setas movem 1U; Delete remove.
- **Fotos reais**: envie a foto do painel frontal; a altura em U se ajusta à proporção da foto, sem cortar. "Usar nos outros do modelo" aplica a foto em todos iguais. Botão **Fotos** mostra/esconde.
- Frente/traseira, profundidade (ambas, só frente, só traseira), estado (instalado, planejado, a remover), portas e cabos físicos entre equipamentos e entre racks (etiquetas na lateral).

Visão Datacenter

- Botão **Datacenter**: todos os racks do mesmo local lado a lado, **uma linha por fileira**, sobre piso elevado, com etiqueta do nome, barra de ocupação (verde, laranja >60%, vermelha >85%) e ocupação total da fileira. Abre enquadrado; clique num rack para abri-lo.
- **Nomes ao lado**: nomes à esquerda com seta, como nas plantas.
- **Zoom pela roda do mouse** e arrastar o fundo, como no diagrama.

Exportação dos racks

PNG, SVG e PDF do rack aberto ou da planta Datacenter; **PDF de todos** (um rack por página); **Relatório** .xlsx com racks, ocupação, U livres, inventário e cabos.

Código: `assets/js/recursos/racks.js`; regras físicas e catálogo (`CATEGORIAS_RACK`, `MODELOS_RACK`) em `assets/js/comum/esquema.js`; fotos em `assets/images/racks/`.

10. Documentos (manuais e procedimentos)

- Menu **Documentação** → **Documentos**. Lista por pasta à esquerda, conteúdo à direita, busca.
- **+ Procedimento**: texto escrito no Grafo, com formatação simples: `# Título`, `- item`, `1. passo`, `**negrito**`, `` `comando` ``. Fica dentro do projeto (vai junto no JSON e na nuvem).
- **Enviar arquivo** (PDF, imagem, texto, até 50 MB): vai para o Supabase Storage (bucket `documentos`); abre por link temporário de 10 minutos.
- **Ligado a**: projeto inteiro, um equipamento, ou um **modelo** (todos os equipamentos daquele modelo mostram o documento).
- A paleta Ctrl+K encontra documentos pelo título e conteúdo.

> **Configuração necessária para arquivos**
> Crie o bucket `documentos` (privado, 50 MB) e rode `supabase/migracoes/2026-10-15-documentos.sql`. Sem isso, os procedimentos funcionam e o envio de arquivo mostra um aviso explicando.

Código: `assets/js/recursos/documentos.js`; campos em `CAMPOS_DOCUMENTO` (`esquema.js`); API `documentosDoProjeto` / `salvarDocumentos` em `src/app/10`.

11. Exportação

Formato	Onde e como
PDF para imprimir	Menu **Exportar** → **PDF para imprimir...**: formato A0–A5, orientação, escala (ajustar ou 25–200% em várias páginas), margem, cabeçalho e legenda, com prévia. Vetorial (texto nítido e pesquisável); se o gerador não carregar, sai como imagem.
PNG / SVG	Menu Exportar. Com cabeçalho (projeto, data, totais) e legenda.
Tema escuro	No tema escuro a exportação sai escura, com as cores da tela (textos e portas claros, ícones clareados). Para imprimir em branco, troque para o tema claro antes.
Projeto (.json)	Exportar → **Exportar projeto** (Ctrl+S), **Importar projeto**, **Baixar todos os projetos** (backup completo).
JSON e CSV	Menu Integração → **JSON e CSV**: editar o projeto em JSON, importar planilha CSV (uma linha por cabo; colunas origem, destino, portas, IPs, VLAN, modelo, tipo, capacidade) e baixar o modelo CSV.

Código: `src/app/11-exportacao-svg-png.js`, `12-pagina-e-pdf.js`, `09-csv-e-mensagens.js`.

12. Relatórios e etiquetas

- **Relatório de inventário** (.xlsx): equipamentos, IPs, cabos e portas livres/em uso. Gerado no navegador (`planilha.js`).
- **Etiquetas com QR code**: PDF em folhas A4 de etiquetas (63,5×38,1, 99,1×38,1 ou 38,1×21,2 mm). Equipamentos com QR code que abre o Grafo direto no equipamento; cabos com duas etiquetas (uma por ponta). Imprima em 100%.

13. Equipe, compartilhar e histórico

- **Equipe**: transforma o projeto em projeto de equipe com link de convite; alterações em tempo real; o dono gerencia membros e o link.
- **Compartilhar**: link só para visualizar (sem login), atualizado sozinho; pode ser desativado na hora.
- **Histórico**: versões automáticas (no máximo uma a cada 10 minutos, por 90 dias); restaurar e recuperar projeto excluído.

14. NetBox e status online

- Menu Integração → **NetBox e status online**: carregar POPs, gerar o diagrama de um POP a partir do cabeamento do NetBox, verificar online (ping) e verificar a cada 60 s.
- Precisa do **agente** rodando no mesmo computador: `cp agente/.env.exemplo agente/.env` (NETBOX_URL e token só leitura) e `npm run agente` (porta 8787).
- Para testar sem NetBox: `npm run netbox-falso` (porta 8000, token `teste123`).

15. Paleta de comandos, atalhos e ajuda

- `Ctrl` `K` (ou o botão **Buscar...** no topo): busca equipamentos (abre o editor), projetos, documentos e ações (exportar, organizar, inventário, etiquetas, tipo de diagrama, tema...). Ignora acentos; setas, Enter e Esc.
- `?` abre a folha de atalhos.

Atalho	Ação
Ctrl+K	Paleta de comandos
Ctrl+S	Exportar projeto (.json)
Ctrl+Z / Ctrl+Y	Desfazer / refazer
Ctrl+C / Ctrl+V / Ctrl+D	Copiar / colar / duplicar seleção
Ctrl+A	Selecionar todos
Delete	Excluir seleção (com Desfazer)
Enter / F2	Editar o equipamento em foco
Setas	Mover o equipamento em foco
Alt (arrastando)	Arrastar sem encaixar nas guias
Esc	Fechar janela ou limpar seleção

Código: `assets/js/recursos/comandos.js` (paleta, ficha do equipamento, folha de atalhos, galeria); atalhos de edição em `src/app/03`.

16. Temas e aparência

- Botão de tema no topo: Automático, Claro, Escuro e temas extras (Nord, Dark Modern, Dracula, Dark+, Catppuccin Mocha, GitHub Light, Solarized Light).
- Fonte dos títulos: Space Grotesk; texto: IBM Plex Sans/Mono.
- Menu lateral em cartão de altura fixa, alinhado ao painel do diagrama; recolhível.
- Avisos com ícone e barra de tempo; esqueleto animado enquanto o diagrama carrega; animações respeitam "reduzir movimento" do sistema.

Código: `assets/css/styles.css` (variáveis de tema no início; blocos de acabamento datados no fim); temas em `src/app/07-temas.js`.

17. Sem internet e app instalável

- Depois de entrar uma vez com internet, o Grafo abre no mesmo aparelho sem conexão: ver, editar e exportar. As alterações sobem quando a internet volta (com aviso de conflito se for o caso).
- Instalável como aplicativo (botão "Instalar app" ou menu do navegador). Arquivos: `sw.js` e `manifest.webmanifest`. Ao mudar a forma de guardar, troque o nome do cache em `sw.js`.

18. Estrutura do código

```
index.html, ver.html          app (login + editor) e link compartilhado (/ver?t=...)
manifest.webmanifest, sw.js   app instalável e sem internet
assets/css/styles.css         estilos, temas e animações
assets/images/                app/ (logo), equipamentos/ (ícones), racks/ (fotos reais)
assets/js/editor/             app.js e icones-embutidos.js  (GERADOS)
assets/js/conta/              auth.js (login, sincronização), sessao.js
assets/js/comum/              esquema.js (validação), erros.js, interface.js (menus, dicas)
assets/js/recursos/           racks, documentos, comandos, equipe, compartilhar, historico,
                              etiquetas, planilha, integracao, primeiros-passos, instalar
assets/vendor/                Zod
src/app/00..12-*.js           partes do editor  (É AQUI QUE SE EDITA)
data/equipamentos.json        tipos e modelos de equipamento (portas)
api/                          funções da Vercel (catálogo, configuração do Supabase)
agente/                       agente local do NetBox/ping e NetBox falso
supabase/                     schema.sql e migracoes/
scripts/                      montar-app.js, gerar-icones-embutidos.js, servidor-local.js
tests/unidade, tests/navegador testes
```

Parte do editor (src/app)	Conteúdo
00-catalogo	Ícones, símbolos, catálogo de modelos e portas
01-estado-e-vista	Estado, elementos da página, zoom e arraste da vista
02-posicoes-e-portas	Posições absolutas e lado das portas
03-cabos-grupos-e-arraste	Roteamento dos cabos, grupos, arrastar, guias, seleção, atalhos, anotações
04-busca-caminho-e-limpar	Busca, rastrear caminho, limpar diagrama
05-zoom-modais-e-editor	Zoom, toque, barra lateral, modais, painel de edição
06-projetos-historico-e-backup	Projetos, tipos de diagrama, transição, tela vazia, backup
07-temas	Temas
08-listas-e-formularios	Editar em lista, formulários, cores por tipo
09-csv-e-mensagens	CSV, avisos (com Desfazer) e erros
10-diagrama-e-layout	DOT, imagens no SVG, organizar, renderização, `window.GrafoApp`
11-exportacao-svg-png	SVG, PNG, moldura, papel escuro na exportação
12-pagina-e-pdf	Página e PDF

API para módulos: `window.GrafoApp`

`projetoAtual`, `nomesProjetos`, `dadosProjeto`, `trocarProjeto`, `editarEquipamento`, `idDoNo`, `avisar`, `svgDoDiagrama`, `racksDoProjeto` / `salvarRacks`, `documentosDoProjeto` / `salvarDocumentos`, `equipamentosDoDiagrama`, `bibliotecasPdf`, `baixarArquivo`, `importarProjetoExterno`, entre outras. Os módulos são carregados por `auth.js` (`loadApp`) depois do editor; ao criar um módulo novo, adicione-o lá e na lista de arquivos guardados para uso sem internet (`keepEditorOffline`).

19. Rodar, testar e publicar

Rodar localmente (Node.js 18+; testes pedem 22+)

```
npm install
copy .env.example .env        # preencha SUPABASE_URL e SUPABASE_PUBLISHABLE_KEY
npm start                     # http://localhost:3000
```

Comando	O que faz
`npm start` / `npm run dev`	Servidor local (dev reinicia ao salvar o servidor)
`npm run montar`	Gera `assets/js/editor/app.js` a partir de `src/app/`
`npm run icones`	Gera os ícones embutidos para a exportação
`npm test`	Testes de unidade (75): validação, racks, documentos, segurança, montagem...
`npm run test:navegador`	Testes num navegador de verdade (48), com Supabase simulado; usam Edge/Chrome instalados
`npm run agente` / `netbox-falso`	Agente local do NetBox e NetBox de teste

Fluxo de trabalho

1. Edite (`src/app/` , `assets/` , `index.html`).
2. `npm run montar` se mexeu em `src/app/` .
3. `npm test` e `npm run test:navegador` : **publique só com tudo verde**.
4. `git commit` e `git push origin main` : a Vercel roda `npm test` e publica. Se falhar, o site continua na versão anterior.
5. Conferir o deploy: `gh api repos/iago069/Grafo-project/commits/<sha>/statuses` (contexto "Vercel – thegrafo").

Variáveis na Vercel

`SUPABASE_URL` e `SUPABASE_PUBLISHABLE_KEY` em Settings → Environments (Production e Preview); depois **Redeploy**. Conferir em `/api/auth-config` .

20. Supabase: configuração completa

1. SQL Editor: rodar `supabase/schema.sql` .
2. Migrações em `supabase/migracoes/` , uma vez cada, na ordem: excluir-conta, compartilhar, equipes, histórico, erros e **documentos** (este depois de criar o bucket).
3. Authentication: e-mail ligado, "Confirm email" ligado em produção; Site URL e Redirect URLs com o endereço do app e `http://localhost:3000` .
4. Storage: bucket `documentos` , privado, 50 MB por arquivo.
5. Erros dos usuários: Table Editor → `grafo_erros` , ou `select * from grafo_erros_resumo;` .

21. Segurança

- **Nunca** use a chave `secret` / `service_role` no front, no `.env` ou na Vercel; só a publishable. A proteção vem do RLS nas tabelas e das policies do Storage.
- O `.env` não vai para o Git.
- CSP no `vercel.json` (e no servidor local): scripts só do próprio site e do jsDelivr, sem script embutido no HTML, sem abrir em iframe de outro site; fontes do Google Fonts. Bibliotecas do CDN com versão fixa e `integrity`. O teste `seguranca.test.js` confere.
- Fotos dos racks ficam no projeto como data URL (até ~250 KB cada); arquivos grandes vão para o Storage.
- O agente só aceita origens configuradas em `ORIGENS` e pode exigir `AGENTE_CHAVE`; use token de NetBox só leitura.

22. Limites e problemas comuns

Limite	Valor
Equipamentos por diagrama / cabos	35 / 2000
Projetos por conta	100
Racks por projeto / itens por rack / cabos físicos	60 / 120 / 3000
Documentos por projeto / texto do procedimento	300 / 20 000 caracteres
Arquivo enviado	50 MB (e o espaço do plano Supabase: 1 GB no gratuito)

Sintoma	Solução
"Configure SUPABASE_URL..."	Variáveis no `.env` ou na Vercel (com Redeploy)
Link do e-mail abre localhost	Ajustar Site URL e Redirect URLs no Supabase
Enviar arquivo avisa que o armazenamento não existe	Criar o bucket `documentos` e rodar a migração de documentos
Teste de montagem falha	Rodar `npm run montar`
Mudança não aparece no site	Ctrl+F5 (cache do service worker)

23. Próximos passos sugeridos

- Sincronizar racks, equipamentos, cabos e IPs do NetBox com o Bayface (NetBox como fonte da verdade, Grafo como camada visual).
- Documentos compartilhados pela equipe (policy por projeto de equipe) e prévia de PDF dentro do app.
- FTTH: caixas de emenda, splitters e orçamento de potência óptica.
- Status ao vivo nos racks (LEDs do Bayface ligados ao ping).
- Domínio próprio, termos de uso e política de privacidade (LGPD), monitoramento de erros (Sentry) e ambiente de testes separado.